KEPLER

Planetary Image Processing Suite

Technical Reference

Version 1.1.5

Architecture Overview

Kepler is written in Python 3 using the tkinter GUI toolkit. It is structured as three modules:

Parameter / Name	Type	Description
main.py	Entry point	Launches the application, sets up the crash handler, and calls app.App().
app.py	GUI + pipeline	All tkinter widgets, layout, preview rendering, histogram, and live pipeline orchestration. Approximately 2,200 lines as of v1.1.6.
processing.py	Algorithms	All numerical image processing: wavelet sharpening, FFT denoising, color adjustments, De-rind, batch alignment. Pure numpy/scipy — no GUI dependencies.

Processing Pipeline

Every slider change triggers a full re-run of the pipeline on the original loaded array. No incremental updates are applied. The pipeline runs in a background thread to keep the GUI responsive.

```
original_arr
│
├─ Wavelet sharpen  (processing.wavelet_sharpen)
│   └─ optional PRE-FFT per layer
│
├─ POST FFT denoise  (processing.fft_denoise)  [if enabled]
│
├─ Color adjustments  (processing.apply_color_adjustments)
│
└─ De-rind  (processing.apply_derind)  [if enabled]
    │
    └─ working_arr  →  preview / export / batch
```

Dependencies

Parameter / Name	Type	Description
numpy ≥ 1.21	Required	Core array operations. All images stored as float32 arrays in [0, 1].
Pillow (PIL) ≥ 9.0	Required	Image loading (8-bit TIFF, PNG, JPEG) and saving. Also used for batch PNG/JPEG output and animated GIF export.
tifffile ≥ 2022	Required	16-bit TIFF loading and saving with full bit-depth preservation.

Parameter / Name	Type	Description
scikit-image ≥ 0.19	Recommended	CLAHE implementation (equalize_adapthist). A scipy fallback is used if not installed, but skimage gives better performance and quality.
scipy.ndimage ≥ 1.7	Required	gaussian_filter, median_filter, binary_dilation, distance_transform_edt, sobel — used throughout sharpening and De-rind.
scipy.optimize ≥ 1.7	Required	curve_fit — used in Auto Denoise power-law fitting.
tkinter	stdlib	GUI toolkit. Included with the standard Python distribution on all platforms.

Image Loading and Internal Format

File Loading

16-bit TIFF: Loaded via tifffile.imread(). The raw uint16 array is divided by 65535.0 to produce a float32 array in [0, 1]. This preserves all 16 bits of dynamic range throughout processing.

All other formats: Loaded via PIL Image.open(), converted to RGB mode, then to float32 via pil_to_array().

```
def pil_to_array(img):
    raw = np.array(img)
    if raw.dtype == np.uint16:
        return raw.astype(np.float32) / 65535.0
    elif raw.dtype == np.uint8:
        return raw.astype(np.float32) / 255.0
    else:
        return (raw - raw.min()) / (raw.max() - raw.min() + 1e-9)
```

Internal Array Format

All images are stored as numpy float32 arrays of shape (H, W, 3) with values in [0.0, 1.0]. Channel order is RGB throughout. No premultiplied alpha. No clipping is applied until export.

TIFF Export

16-bit TIFF export uses tifffile.imwrite() with dtype uint16. The float32 array is multiplied by 65535.0 and clipped before conversion. This preserves full dynamic range and is not routed through PIL (which would truncate to 8-bit).

Wavelet Sharpening Algorithm

The core sharpening algorithm is implemented in processing.py. The public entry point is wavelet_sharpen(). Internally it calls _sharpen_plane() per channel or per luma pass, depending on the Convolve mode.

Gaussian Mode Calibration

Kepler's Gaussian mode has been carefully calibrated for optimal planetary sharpening. Key operating points:

- Sigma ≈ 0.5 px (sub-pixel Gaussian kernel) at Layer 1 — captures the finest resolvable detail.
- At slider = 200, approximately 350× gain on sigma = 0.5 px USM detail — the calibrated reference operating point.
- gain_div_L1 = 200/350 ≈ 0.5714 so slider 200 → gain 350× on L1.
- Per-layer scale factor = 4: gain_div_L2 = 2.2857, gain_div_L3 = 9.1429.

Note: The scale factor of 4 gives all three layers clearly visible and independently controllable effects across the full slider range. An earlier version used scale = 16, which made L2 and L3 essentially invisible at slider = 200.

Color Models

Oklab (default)

The image is converted from sRGB to the Oklab perceptually uniform color space (Björn Ottosson, 2020). The L channel is extracted for sharpening. After sharpening, L is replaced and the image is converted back to sRGB.

```
# sRGB → linear RGB
linear = ((c + 0.055) / 1.055) ** 2.4  for c > 0.04045

# linear RGB → LMS (M1 matrix)
l = 0.4122*r + 0.5363*g + 0.0514*b
m = 0.2119*r + 0.6807*g + 0.1074*b
s = 0.0883*r + 0.2817*g + 0.6300*b

# cube root then LMS → Lab (M2 matrix)
L = 0.2105*l_ + 0.7936*m_ - 0.0041*s_
```

HSL and HSV

Alternative color models. Less accurate than Oklab for planetary work due to hue instability near saturated pixels.

Gaussian Mode (default)

Multi-scale USM. For each layer i with sigma σ_i:

```
# L1 (i=0): standard USM
```

```
detail_L1 = plane - gaussian_filter(plane, σ_0)

# L2, L3 (i>0): bandpass (prevents double-boosting sub-σ content)
detail_Li = gaussian_filter(plane, σ_{i-1}) - gaussian_filter(plane, σ_i)

# SharpenFilter DoG boost (applied to L1 only in practice):
dog_i = smoothed_i - gaussian_filter(plane, σ_i * 1.6)
detail_i += sharp_filter * dog_i

# Amplification:
gain_div_i = 0.5714 * (4.0 ** i)   # L1=0.5714, L2=2.2857, L3=9.1429
amp_i = (sharpen_i / gain_div_i) * detail_i * background_mask
output += amp_i
```

Note: L2 and L3 use a true bandpass (smoothed_{i-1} − smoothed_i) rather than standard USM. This prevents L2 from re-sharpening sub-sigma content already boosted by L1, which would cause approximately 4× over-contribution at mid-scale.

Gain Divisor Calibration

Parameter / Name	Type	Description
Gaussian L1 (i=0)	gain_div = 0.5714	slider 200 → gain 350× (matches WS at σ=0.5px)
Gaussian L2 (i=1)	gain_div = 2.2857	slider 200 → gain 87.5×
Gaussian L3 (i=2)	gain_div = 9.1429	slider 200 → gain 21.9×
Bilateral (all layers)	same as Gaussian × 4^i	radius=10 → identical to Gaussian; radius=1 → edges preserved
Z-Gaussian factor=0	gain_div × 2.173	need ~33% of Gaussian slider value for same output
Z-Gaussian factor=1	same as Gaussian	identical to Gaussian mode
b3 / other modes	gain_div = 75.0 (flat)	slider 100 ≈ unity gain

Background Mask

A soft foreground mask protects the sky background from receiving sharpening artefacts. Using a fraction of the image peak makes the threshold robust for any planet size (large Jupiter to small Mars):

```
lum_max    = percentile(plane, 99.5)      # robust peak
lo_thresh  = lum_max * 0.015              # 1.5% of peak
```

```
hi_thresh  = lum_max * 0.030             # 3.0% of peak
mask       = clip((plane - lo_thresh) / (hi_thresh - lo_thresh), 0, 1)
mask       = gaussian_filter(mask, sigma = H / 60)
```

Bilateral Mode

Edge-preserving filter. Uses Decomp σ for spatial smoothing (same as Gaussian), but blends the result using a per-pixel weight derived from Sobel edge magnitude — flat areas receive full Gaussian smoothing, strong edges are preserved.

```
smooth    = gaussian_filter(plane, sigma)         # Decomp σ
edge_mag  = hypot(sobel(plane, x), sobel(plane, y))
edge_scale = percentile(edge_mag, 99.0)

# Logarithmic threshold from Radius slider (1-10):
threshold = edge_scale × (0.3 × (100/0.3) ^ ((radius-1)/9))

ew = exp(-(edge_mag² / (2 × threshold²)))
result = plane × (1-ew) + smooth × ew

# radius=1:  ew→0 at limb/belts → plane preserved there
#            ew→1 in flat areas  → full Gaussian in disk interior
# radius=10: ew=1 everywhere    → identical to Gaussian
```

bilateral_layers: list of bool per layer. When a layer's flag is False, Gaussian is used for that layer regardless of the global filter setting. gain_div uses the Gaussian schedule (0.5714 × 4^i) so slider values are identical to Gaussian at radius=10.

Z-Gaussian Mode

Wider sigma schedule than Gaussian. Controlled by the Factor parameter (0–1):

```
# Sigma blend: factor=0 → full ZG sigma; factor=1 → Decomp σ (Gaussian)
sigma = sigma_zg + factor × (sigma_gauss - sigma_zg)
  where sigma_zg = 1.4 × 2^scale

# Gain divisor blend: factor=0 → 2.173× Gaussian gain_div
#                     factor=1 → identical to Gaussian
gain_div_mult = 2.173 × (1 - factor) + 1.0 × factor
gain_div_i    = (200/350) × gain_div_mult × (4.0 ^ i)
```

At factor=0, slider ~67 produces the same visual output as Gaussian slider 200 (empirically calibrated: 2.2% difference). At factor=1, output is identical to Gaussian.

Convolve Modes

L — Luminance Only

Extracts the L channel, sharpens it, substitutes the sharpened L back, converts to RGB. Color information preserved exactly.

RGB — Per-Channel

Runs _sharpen_plane() independently on R, G, B. Can produce slight color fringing on high-contrast edges.

LRGB — Per-Channel RGB (default)

Sharpens each of R, G, and B independently. Produces the strongest overall sharpening result. The two-pass approach (L then RGB) was removed after pixel-level analysis showed it diluted the gain.

FFT Denoising Algorithm

Implemented in processing.fft_denoise(). Applies a radially symmetric low-pass filter in the 2D discrete Fourier domain. Operates independently on each RGB channel.

```
for each channel c:
    F = fftshift(fft2(image[c]))
    dist = sqrt((x-cx)² + (y-cy)²) / max_dist   # normalized 0-1
    filter = rolloff_curve(dist, start_frac, width_frac, curve_k)
    F *= filter
    output[c] = real(ifft2(ifftshift(F)))
```

Frequency Normalisation

max_dist = min(cx, cy) — the Nyquist distance (half the minimum image dimension). This is critical: using the corner distance sqrt(cx^2+cy^2) would make the frequency markers 1.414× too large (√2 error), causing the filter to be far too wide. At user settings start=20%, end=70%, the corner-distance version would effectively start at 28% and end at 99% of Nyquist, leaving almost nothing removed.

Rolloff Curve

The curve slider (0–100) maps to a power exponent k via: k = 0.3 × (10/3)^(curve/50). At curve=0 k≈0.30 (gentle), curve=50 k=1.0 (linear), curve=100 k≈3.33 (aggressive).

```
# Within the rolloff ramp [start, end]:
t      = (nd - start_frac) / (end_frac - start_frac)   # 0 at start, 1 at end
filter = (1.0 - t) ** k
```

Auto Denoise Algorithm

Implemented in app._fft_auto_denoise(). Uses the power spectrum of the working image (post-wavelet):

- Smooth the 64-bin radial power spectrum with a σ=2 Gaussian.
- Compute the first derivative (slope per bin).
- Find the initial drop rate from the first 8 bins (signal-dominated region).
- Find the first bin where the slope slows to less than 10% of the initial drop rate — this marks where signal energy gives way to noise.

- Set Start to that bin position (clamped to 15–45% Nyquist). End is fixed at 70%.
- Curve is estimated from the plateau flatness between 30–70%.

Tools — Deconvolution, Dehaze, Levels & Curves and CLAHE

Pipeline position: after color adjustment, before de-rinding. Each step is a no-op at default/disabled settings.

Color Denoise

Available when Convolve is RGB or LRGB. Converts the sharpened result to oklab and applies a Gaussian blur with sigma=radius to the a* and b* chroma channels only — luminance (L) is unchanged. Suppresses color noise amplified by RGB/LRGB sharpening.

```
_apply_color_denoise(img, radius) -> float32
  lab = rgb_to_oklab(img)
  lab[...,1] = gaussian_filter(lab[...,1], sigma=radius)  # a* chroma
  lab[...,2] = gaussian_filter(lab[...,2], sigma=radius)  # b* chroma
  return oklab_to_rgb(lab)
```

Deconvolution

Richardson-Lucy deconvolution on the oklab L channel using a 5×5 Moffat PSF (gamma=1, beta=2). One RL iteration applied, then optional contrast weighting using local std dev to protect flat areas from over-sharpening.

```
PSF(r) = (1 + (r/gamma)^2)^(-beta)        # Moffat seeing kernel
conv        = PSF ⊗ L                       # forward convolve
relative    = L / (conv + ε)               # pixel-wise ratio
corr_delta  = (PSF_mirror ⊗ relative) - 1  # RL correction
weight      = contrast_norm^power          # if use_contrast
L_out       = clip(L + (strength/3.14) * weight * corr_delta * L)
```

Dehaze

Edge-based chroma suppression targeting chromatic fringing at planetary limbs. Detects high-gradient zones on luminance using Sobel, spreads the mask by block_size × 0.4 sigma, then blends each RGB channel toward luminance at those locations.

```
edge_mag    = sqrt(sobel_x^2 + sobel_y^2)  # on Gaussian-smoothed luma
fringe_mask = gaussian_filter(edge_norm, sigma=block_size*0.4)
result[c]   = img[c] + fringe_mask * amount * (lum - img[c])
```

At fringe pixels: amount=1 pulls the channel fully to neutral gray (lum). At non-edge pixels: fringe_mask ≈ 0 so the image is unchanged. Chromatic aberration (e.g. blue limb fringing) appears as a color offset from luma — this corrects it directly.

Levels & Curves

Per-channel black/white point stretch followed by gamma and a 256-entry PCHIP spline LUT:

```
stretched = clip((x - black) / (white - black), 0, 1)
gamma_out = stretched ^ (1 / gamma)
lut_out   = curve_lut[int(gamma_out * 255)]
```

The tone curve is built from user-draggable control points using PchipInterpolator (monotone cubic). is_default=True skips processing when all values are at identity.

```
apply_levels_curves(img, black_r, white_r, gamma_r, black_g, white_g, gamma_g,
                    black_b, white_b, gamma_b, curve_lut_r, curve_lut_g,
curve_lut_b)
```

CLAHE

Contrast Limited Adaptive Histogram Equalisation via skimage.exposure.equalize_adapthist, with a scipy fallback using bilinear tile interpolation. Luminance mode scales RGB by the luma ratio; RGB mode applies independently per channel.

```
apply_clahe(img, clip_limit, tile_grid, channel_mode, strength)
  # strength blends: out = img + strength * (enhanced - img)
```

Note: scikit-image is required for best results. Install with: pip install scikit-image

Color Adjustment Algorithm

Implemented in processing.apply_color_adjustments(). Operations are applied in this fixed order:

```
1. Per-channel Black Point (levels stretch)
   out[c] = clip((in[c] - black[c]) / (1 - black[c]), 0, 1)

2. Per-channel Gamma
   out[c] = in[c] ^ (1 / gamma[c])

3. Per-channel Gain
   out[c] = in[c] × gain[c]

4. Brightness (additive offset)
   out = in + brightness

5. Contrast (pivot at 0.5)
   out = (in - 0.5) × (1 + contrast) + 0.5

6. Saturation
   lum = 0.299r + 0.587g + 0.114b
   out = lum + (in - lum) × saturation

7. Vibrance (selective saturation boost)
   vib_boost = (vibrance - 1) × (1 - S)
   S_new = S + vib_boost × S
```

```
8. Hue Rotation
   H = (H + rotation / 360) mod 1.0
```

Slider-to-Pipeline Mapping

The Gain, Saturation, and Vibrance sliders display 0–100 but are internally stored 0–200. The pipeline divides by 100 to convert to the float multiplier. At the displayed neutral value of 50 (internal 100), gain/saturation/vibrance = 1.0 (no change).

Gamma is stored 50–200. The pipeline divides by 100 to get the exponent (0.50–2.00). At 100 (displayed as 1.00), gamma = 1.0 (no change).

De-rind Algorithm

Implemented in processing.apply_derind(). Builds a soft mask around the detected planetary limb and blends masked pixels toward a locally-averaged target.

Disk Detection

All De-rind functions use a threshold of 15% of the image peak (percentile 99.5) to detect the planet disk. This is robust across all planet sizes — from a Moon filling the frame to a small Mars disk — and is always well above sky noise (typically < 1% of peak). Earlier percentile-based approaches (p85 for 'planet level') failed when the disk occupied less than 15% of the image.

```
lum_max    = percentile(lum, 99.5)
disk_thresh = max(lum_max * 0.15, 0.01)
disk_mask   = lum > disk_thresh
```

Mask Geometry

Distance transforms from the detected limb boundary define the mask shape:

```
dist_sky    = distance_transform_edt(~disk_mask)   # sky pixels: dist to limb
dist_planet = distance_transform_edt(disk_mask)    # planet pixels: dist to limb

# Inset shifts the effective limb inward by inset_px:
eff_outer   = max(edge_px - inset_px, 0.0)
outer_ramp  = clip(1.0 - dist_sky / eff_outer, 0, 1)  if eff_outer > 0  else 0

# Inner feather: ramp starts at inset_px inside the limb
inner_ramp  = clip(1.0 - (dist_planet - inset_px) / feather_px, 0, 1)
inner_ramp[dist_planet < inset_px] = 0.0   # not yet inside band

mask = outer_ramp * (sky pixels) + inner_ramp * (planet pixels)
mask = gaussian_filter(mask, smooth_px)
```

Note: At inset = 0 the band starts at the detected limb boundary. As inset increases, the sky-side mask shrinks. At inset >= edge, the sky side is completely suppressed and the band sits entirely inside the disk.

Dark Edge Extension

When Dark Edge is enabled, a second mask is built with edge = edge × 2.5 and merged with the primary mask using np.maximum(mask, outer_ext × 0.5). This produces a single continuous gradient band rather than two separate rings.

Mask Overlay Rendering

When Show De-rind Mask overlay is active, the overlay is rendered per-pixel with no global darkening:

```
tint_strength = clip(mask * 0.90, 0, 1)[..., newaxis]
red_tint      = [1.0, 0.0, 0.0]
result = image * (1.0 - tint_strength * 0.7) + red_tint * tint_strength * 0.7
```

Pixels where mask = 0 (sky and disk interior) are completely untouched. Only masked pixels receive the red tint.

Correction

The correction target is a locally-averaged version of the image at the scale of the edge:

```
kernel  = max(3, int(edge) * 2 + 1)
target  = uniform_filter(image, size=kernel)  # per channel
mask3   = mask[..., newaxis]
result  = image * (1.0 - mask3) + target * mask3
```

Batch Processing

Implemented in app._batch_run(). All pipeline parameters are captured on the main thread before the worker thread launches. This avoids race conditions with slider values during processing.

Batch Worker Thread

For each file the worker: loads the image (tifffile for TIFF, PIL for others), optionally aligns it, runs the full pipeline (wavelet → FFT → color → De-rind), optionally rotates it, then saves it. Progress is reported back to the main thread via self.root.after(0, callback).

Planet Alignment

Detects the disk center using the same 15%-of-peak threshold as De-rind, measures the centroid of the disk mask, and applies a translation shift using scipy.ndimage.shift to align to the first file's center.

```
lum_max  = percentile(lum, 99.5)
disk     = lum > max(lum_max * 0.15, 0.01)
ys, xs   = where(disk)
cy, cx   = mean(ys), mean(xs)   # disk centroid
dy, dx   = round(ref_cy - cy), round(ref_cx - cx)
shifted  = ndimage.shift(channel, (dy, dx), mode='constant', cval=0)
```

Rotation

Applied after the full pipeline per file using PIL Image.rotate(). Positive angle = clockwise. PIL rotate() internally negates the angle (PIL uses CCW convention), so the call uses -deg.

```
pil_img = Image.fromarray(clip(img*255, 0, 255).astype(uint8))
rotated = pil_img.rotate(-deg, expand=expand,
                         resample=Image.BICUBIC, fillcolor=(0,0,0))
```

Animation Export

_build_animation() creates animated WebP or GIF from a list of image file paths:

- WebP: PIL Image.save() with save_all=True, quality=90, duration in milliseconds, loop=0 (continuous).
- GIF: Frames are color-quantised to 256 colors using median-cut (PILImage.Quantize.MEDIANCUT) before saving.

processing.py — Public API

All functions take and return numpy float32 arrays of shape (H, W, 3) with values in [0, 1] unless otherwise noted.

wavelet_sharpen()

```
wavelet_sharpen(
    img_array,          # float32 (H,W,3) input image
    levels,             # list of (sharpen, denoise, sigma) per layer
    filter_method,      # 'gaussian'|'zgaussian'|'bilateral'|'b3'
    color_model,        # 'oklab'|'hsl'|'hsv'
    convolve,           # 'L'|'RGB'|'LRGB'
    pre_fft,            # list of (start,width,curve)|None per layer
    sharp_filter,       # float or list[float] — DoG coefficient per layer
    pre_smooth,         # float — pre-decomposition Gaussian sigma
    power_fn,           # float — power exponent (1.0 = off/linear)
    bilateral_radius,   # float 1-10 — edge threshold (10=identical to Gaussian)
    zgauss_factor,      # float 0-1 — ZG strength (0=full ZG ~33% sliders,
1=Gaussian)
    bilateral_layers,   # list[bool]|None — per-layer bilateral enable flags
) -> float32 (H,W,3)
```

fft_denoise()

```
fft_denoise(
    img_array,   # float32 (H,W,3)
    fft_start,   # float 0-100, % of Nyquist where rolloff starts
    fft_width,   # float 0-100, width of rolloff (end - start)
    fft_curve,   # float 0-100 — curve shape slider
) -> float32 (H,W,3)
```

apply_color_adjustments()

```
apply_color_adjustments(
    img_array,
    r_gain, g_gain, b_gain,       # float 0-2  (slider/100)
    r_gamma, g_gamma, b_gamma,    # float 0.5-2.0  (slider/100)
    saturation,                   # float 0-2  (slider/100)
    vibrance,                     # float 0-2  (slider/100)
    hue_rotation,                 # float degrees -180..+180
    brightness,                   # float additive offset
    contrast,                     # float scale factor
    r_black, g_black, b_black,    # float 0-1 black point
) -> float32 (H,W,3)
```

apply_derind()

```
apply_derind(
    img_array,
    edge,          # float px — outer extent beyond limb
    feather,       # float px — inward falloff depth
    smooth,        # float px — mask Gaussian sigma
    inset,         # float px — pull mask inward from limb
    gap_width,     # float degrees — angular gap (0 = none)
    gap_angle,     # float degrees — gap center angle
    saturn_mode,   # bool — symmetric 180° gaps
    dark_edge,     # bool — extend mask outward for diffraction rings
    pre_blur,      # float sigma — pre-blur before correction
    show_ring_map, # bool — overlay mask as red tint
) -> float32 (H,W,3)
```

Utility Functions

Parameter / Name	Type	Description
pil_to_array(img)	→ float32	Converts PIL Image to float32 [0,1] array. Handles 8-bit and 16-bit inputs.
array_to_pil(arr)	→ PIL.Image	Converts float32 [0,1] array to uint8 PIL Image for display or JPEG/PNG export.
luminance(arr)	→ float64	Returns Rec.601 luma: 0.299r + 0.587g + 0.114b.

Parameter / Name	Type	Description
auto_balance(arr)	→ tuple	Returns (r_gain, g_gain, b_gain, r_black, g_black, b_black) for histogram-based RGB balance.
auto_white_balance(arr)	→ tuple	Returns (gains, gammas) from percentile white-point and midtone stretch per channel.
estimate_filter_sigma(sharpen, level)	→ float	Returns Decomp σ for level l: base=0.5 × 2^l. L1=0.5px, L2=1.0px, L3=2.0px.
estimate_sharpen_filter(sharpen, sigma, level)	→ float	Returns calibrated SharpenFilter for the given layer. WS-calibrated values: 0.146 / 0.100 / 0.100.
fft_denoise(img, start, width, curve)	→ float32	Applies radial low-pass FFT filter.
build_fft_filter_curve(start, width, curve, n_bins)	→ float64[64]	Returns 1D filter magnitude profile for the FFT curve display widget.
compute_fft_power(arr)	→ float64[64]	Returns radially-averaged log power spectrum for the FFT display overlay.

app.py — GUI Architecture

Layout Structure

```
root (Tk)
├── header frame — icon, title, status, Reset Layout button
└── _main_pane (PanedWindow, horizontal)
    ├── left panel (min 400 px)
    │   ├── top strip: Open Image / Export Image buttons
    │   └── _vert_pane (PanedWindow, vertical)
    │       ├── preview pane — side-by-side canvases (original | processed)
    │       ├── magnifier pane — StaticMagnifier widget
    │       └── histogram pane — live luminance histogram
    └── right panel (min 500 px)
        └── tab notebook: Wavelet | FFT | RGB | De-rind | Stats | Batch
```

Pipeline Trigger

The method _fire_live() is called by every slider's release callback. It collects all current slider values, packages them into a pipeline parameters object, and submits the job to a background thread via threading.Thread. A cancel token prevents stale results from rendering when sliders are moved quickly.

Preview Rotation

A Rotate spinbox row in the PREVIEW section lets users rotate the displayed result without re-running the pipeline. The rotation is stored in self._preview_rot_deg (DoubleVar).

All paths that draw to cnv_proc go through _get_processed_pil(), which applies the current rotation to the PIL image before display. This ensures zoom, pan, and pipeline redraws all respect the rotation.

```
def _get_processed_pil(self):
    pil = proc.array_to_pil(self.working_arr)
    deg = float(self._preview_rot_deg.get())
    if deg != 0.0:
        pil = pil.rotate(-deg, expand=False, resample=Image.BICUBIC)
    return pil
```

Export: both TIFF 16-bit (via tifffile) and PIL-based exports apply the rotation before saving. working_arr is never modified — the rotation only affects display and export.

Bidirectional sync: _sync_preview_rot_to_batch() mirrors the preview rotation into self._batch_rot_deg and enables self._batch_rotate_on. _sync_batch_rot_to_preview() mirrors changes from the Batch tab back to the preview spinbox and redraws.

LabeledSlider

All sliders use the LabeledSlider class, which wraps tk.Scale with:

- Resolution: Automatically derived from the format string ({:.1f} → resolution 0.1).
- Callable fmt: The format parameter accepts either a format string or a callable (e.g. lambda v: f'{v/2:.0f}') for custom value display. Used by Gain, Saturation, Vibrance (display v/2) and Gamma (display v/100).
- Drag suppression: The callback fires only on mouse release, not during drag.
- Reset: Right-click or double-click resets the slider to its constructor default value.

Design Constants

```
# Backgrounds
BG_DEEP   = '#f0f4f8'   # root window background
BG_PANEL  = '#e2e8f0'   # side panel background
BG_CARD   = '#ffffff'   # control cards
BG_RAISED = '#cbd5e1'   # raised elements / slider troughs

# Accents (per tab)
ACCENT_C  = '#013958'   # blue   (Wavelet)
ACCENT_O  = '#6a2306'   # orange (FFT)
ACCENT_G  = '#0b4621'   # green  (RGB / Batch)
ACCENT_P  = '#441f82'   # purple (De-rind)

# Fonts
F_TINY = ('Consolas', 11)
F_SM   = ('Consolas', 12)
F_MD   = ('Consolas', 13)
F_BOLD = ('Consolas', 15, 'bold')
F_HDR  = ('Consolas', 19, 'bold')
```

Installers

Windows — install_gui.py / install.bat

A tkinter-based installer GUI. Checks for Python 3 on PATH, installs pip packages from requirements.txt, creates a desktop shortcut pointing to main.py, and optionally registers a Start Menu entry.

Shortcut Creation

create_shortcut() tries four methods in order to maximize reliability:

- ctypes IShellLink COM — pure Python, calls Windows' own IShellLinkW COM interface directly via ctypes. Sets TargetPath, Arguments, WorkingDirectory, Description, and IconLocation. Uses the correct vtable indices: SetDescription=7, SetWorkingDirectory=9, SetArguments=11, SetIconLocation=17, SetPath=20. IPersistFile::Save at index 6.
- win32com.client — if pywin32 is installed.
- PowerShell .ps1 file — ExecutionPolicy Bypass, available on all Windows 7+ systems.
- cscript VBS — legacy last resort.

Shortcut creation is scheduled on the main thread via self.after(0, callback) to satisfy the COM STA (Single-Threaded Apartment) threading requirement. Calling COM STA objects from a background thread silently fails.

Windows Version Detection

platform.release() returns '10' for both Windows 10 and Windows 11. The installer uses platform.version() to extract the build number: build >= 22000 = Windows 11, build >= 10240 = Windows 10. The edition (Pro, Home, Enterprise) is read from HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\EditionID.

Linux — install_linux.py / install.sh

A tkinter-based installer GUI for Linux. Detects the system Python 3, installs dependencies into the user site-packages (pip install --user), and creates a .desktop launcher file in ~/.local/share/applications/.

The desktop entry receives the actual installed icon path (install_dir/icon.ico or icon.png) rather than the generic 'python3' fallback. The installer window sets its own icon using wm_iconphoto with PIL.

Quick Reference — Default Slider Values

Setting	Default	Range	Tab
Layer 1 Sharpen	0	0–200	Wavelet
Layer 2 Sharpen	0	0–200	Wavelet

Setting	Default	Range	Tab
Layer 3 Sharpen	0	0–200	Wavelet
Layer 1 SharpenFilter	0.146	0.0–2.0	Wavelet
Layer 2 SharpenFilter	0.100	0.0–2.0	Wavelet
Layer 3 SharpenFilter	0.000	0.0–2.0	Wavelet
Bilateral Radius	2.0	1–10	Wavelet (bilateral only)
Z-Gaussian Factor	1.0	0.0–1.0	Wavelet (zgaussian only)
Note: Decomp σ and Denoise sliders hidden from UI but remain wired			Wavelet
Pre-Smooth σ	0.0	0.0–5.0	Wavelet
Convolve	LRGB	L/RGB/LRGB	Wavelet
FFT Enable	Off	On/Off	FFT
FFT Start %	60	0–100	FFT
FFT End %	90	0–100	FFT
R/G/B Gain (displayed)	50	0–100	RGB
R/G/B Gamma (displayed)	1.00	0.50–2.00	RGB
R/G/B Black Point	0	0–100 %	RGB
Saturation (displayed)	50	0–100	RGB
Vibrance (displayed)	50	0–100	RGB
De-rind Enable	Off	On/Off	De-rind
Edge	8 px	1–40	De-rind
Feather	4 px	0–30	De-rind
Smooth	1	0–20	De-rind
Inset	0 px	0–60	De-rind